

Partitioning

From [Wikipedia](#):

Disk partitioning or disk slicing is the creation of one or more regions on [secondary storage](#), so that each region can be managed separately.

An entire disk may be allocated to a single partition, or multiple ones for cases such as dual-booting, maintaining a [swap](#) partition, or to logically separate data such as audio and video files. The partitioning scheme is stored in a [partition table](#) such as Master Boot Record (MBR) or GUID Partition Table (GPT).

Partition tables are created and modified using one of [many partitioning tools](#). The tools available for Arch Linux are listed in the [#Partitioning tools](#) section.

Partitions usually contain a [file system](#) directly which is accomplished by creating a file system on (a.k.a. [formatting](#)) the partition. Alternatively, partitions can contain [LVM](#), [block device encryption](#) or [RAID](#), which ultimately provide device files on which a file system can be placed (or the devices can be stacked further).

Any [block device](#) (e.g. disk, partition, LUKS device, LVM logical volume or RAID array) that directly contains a mountable file system is called a [volume](#).

Contents [\[hide\]](#)

- 1 [Partition table](#)
 - 1.1 [Master Boot Record](#)
 - 1.1.1 [Master Boot Record \(bootstrap code\)](#)
 - 1.1.2 [Master Boot Record \(partition table\)](#)
 - 1.2 [GUID Partition Table](#)
 - 1.3 [Choosing between GPT and MBR](#)
 - 1.4 [Partitionless disk](#)
 - 1.4.1 [Btrfs partitioning](#)
- 2 [Partition scheme](#)
 - 2.1 [Single root partition](#)
 - 2.2 [Discrete partitions](#)
 - 2.2.1 [/](#)
 - 2.2.2 [/boot](#)
 - 2.2.3 [/home](#)
 - 2.2.4 [/var](#)
 - 2.2.5 [/data](#)
 - 2.2.6 [Swap](#)
 - 2.3 [Example layouts](#)
 - 2.3.1 [UEFI/GPT layout example](#)
 - 2.3.2 [BIOS/MBR layout example](#)
 - 2.3.3 [BIOS/GPT layout example](#)
- 3 [Tools](#)
 - 3.1 [Partitioning tools](#)
 - 3.1.1 [fdisk](#)
 - 3.1.2 [GPT fdisk](#)
 - 3.1.3 [GNU Parted](#)
 - 3.2 [Backup](#)
 - 3.3 [Recovery](#)
- 4 [Partition alignment](#)
- 5 [GPT kernel support](#)
- 6 [Troubleshooting](#)
 - 6.1 [Tricking old BIOS into booting from GPT](#)
- 7 [See also](#)

Related articles

File systems

[fdisk](#)

[gdisk](#)

[parted](#)

[fstab](#)

[LVM](#)

[Swap](#)

[Arch boot process](#)

[Unified Extensible Firmware Interface](#)

Partition table

There are two main types of partition table available. These are described below in the [#Master Boot Record](#) (MBR) and [#GUID Partition Table](#) (GPT) sections along with a discussion on how to choose between the two. A third, less common alternative is using a partitionless disk, which is also discussed.

Use a [partitioning tool](#) to view the partition table of a [block device](#).

Tip: Run `parted /dev/sdX print` or `fdisk -l /dev/sdX`, where `/dev/sdX` is the [block device](#) such as `/dev/sda` for a SATA disk, `/dev/nvme0n1` for a [NVMe](#) disk or `/dev/mmcblk0` eMMC disk. See [Device file#Block device names](#) for more information on block device naming.

Master Boot Record

The [Master Boot Record](#) (MBR) is the first 512 bytes of a storage device. It contains an operating system bootloader and the storage device's partition table. It plays an important role in the [boot process](#) under [BIOS](#) systems. See [Wikipedia:Master boot record#Disk partitioning](#) for the MBR structure.

Note:

- The MBR is not located in a partition; it is located at the first sector of the device (physical offset 0), preceding the first partition.
- The boot sector present on a partitionless device or within an individual partition is called a [volume boot record \(VBR\)](#) instead.

Master Boot Record (bootstrap code)

The first 440 bytes of MBR are the **bootstrap code area**. On BIOS systems it usually contains the first stage of the boot loader. The bootstrap code can be backed up, restored from backup or erased [using dd](#).

Master Boot Record (partition table)

In the MBR partition table (also known as DOS or MS-DOS partition table) there are 3 types of partitions:

- Primary
- Extended
 - Logical

Primary partitions can be bootable and are limited to four partitions per disk or RAID volume. If the MBR partition table requires more than four partitions, then one of the primary partitions needs to be replaced by an **extended** partition containing **logical** partitions within it.

Extended partitions can be thought of as containers for logical partitions. A hard disk can contain no more than one extended partition. The extended partition is also counted as a primary partition so if the disk has an extended partition, only three additional primary partitions are possible (i.e. three primary partitions and one extended partition). The number of logical partitions residing in an extended partition is unlimited. A system that dual boots with Windows will require for Windows to reside in a primary partition.

The customary numbering scheme is to create primary partitions *sda1* through *sda3* followed by an extended partition *sda4*. The logical partitions on *sda4* are numbered *sda5*, *sda6*, etc.

Tip: When partitioning a MBR disk consider leaving at least 33 512-byte sectors (16.5 KiB) of free unpartitioned space at the end of the disk in case you ever decide to **convert it to GPT**. The space will be required for the backup GPT header.

GUID Partition Table

GUID Partition Table (GPT) is a partitioning scheme that is part of the **Unified Extensible Firmware Interface** specification; it uses **globally unique identifiers** (GUIDs), or UUIDs in the Linux world, to define partitions and **partition types**. It is designed to succeed the **Master Boot Record** partitioning scheme method.

At the start of a GUID Partition Table disk there is a **protective Master Boot Record** (PMBR) to protect against GPT-unaware software. This protective MBR just like an ordinary MBR has a **bootstrap code area** which can be used for BIOS/GPT booting with boot loaders that support it.

Choosing between GPT and MBR

GUID Partition Table (GPT) is an alternative, contemporary, partitioning style; it is intended to replace the old Master Boot Record (MBR) system. GPT has several advantages over MBR which has quirks dating back to MS-DOS times. With the recent developments to the formatting tools, it is equally easy to get good dependability and performance for GPT or MBR.

Note: For GRUB to boot from a GPT-partitioned disk on a BIOS-based system, a **BIOS boot partition** is required.

Some points to consider when choosing:

- To dual-boot with Windows (both 32-bit and 64-bit) using Legacy BIOS, the MBR scheme is required.
- To dual-boot Windows 64-bit using **UEFI** mode instead of BIOS, the GPT scheme is required.
- If you are installing on older hardware, especially on old laptops, consider choosing MBR because its BIOS might not support GPT (but **see below** how to fix it).
- If you are partitioning a disk that is larger than 2 TiB, you need to use GPT.
- It is recommended to always use GPT for **UEFI** boot, as some UEFI implementations do not support booting to the MBR while in UEFI mode.
- If none of the above apply, choose freely between GPT and MBR. Since GPT is more modern, it is recommended in this case.

Some advantages of GPT over MBR are:

- Provides a unique disk GUID and unique partition GUID (**PARTUUID**) for each partition - A good filesystem-independent way of referencing partitions and disks.
- Provides a filesystem-independent partition name (**PARTLABEL**).
- Arbitrary number of partitions - depends on space allocated for the partition table - No need for extended and logical partitions. By default the GPT table contains space for defining 128 partitions. However if you want to define more partitions, you can allocate more space to the partition table (currently only *gdisk* is known to support this feature).
- Uses 64-bit LBA for storing Sector numbers - maximum addressable disk size is 2 ZiB. MBR is limited to addressing 2 TiB of space per drive.^[1]
- Stores a backup header and partition table at the end of the disk that aids in **recovery** in case the primary ones are damaged.
- CRC32 checksums to detect errors and corruption of the header and partition table.

The section on **#Partitioning tools** contains a table indicating which tools are available for creating and modifying GPT and MBR tables.

Tip: It is possible to convert between MBR and GPT. See **gdisk#Convert between MBR and GPT**.

Partitionless disk



This article or section needs expansion.



Reason: Explain when one might want to use a partitionless disk (e.g. in VMs) and when not and why. (Discuss in **Talk:Partitioning#**)

Partitionless disk a.k.a. **superfloppy** refers to a storage device without a partition table, having one file system occupying the whole storage device. The boot sector present on a partitionless device is called a **volume boot record (VBR)**.

Btrfs partitioning

Btrfs can occupy an entire data storage device and replace the **MBR** or **GPT** partitioning schemes. See the **Btrfs#Partitionless Btrfs disk** instructions for details.

Partition scheme



This article or section needs expansion.



Reason: Introduce **LVM**, **mdadm**, **dm-crypt**, etc. They could be placed in a new subsection together with the information about Btrfs subvolumes. (Discuss in **Talk:Partitioning#**)

There are no strict rules for partitioning a hard drive, although one may follow the general guidance given below. A disk partitioning scheme is determined by various issues such as desired flexibility, speed, security, as well as the limitations imposed by available disk space. It is essentially personal preference. If you would like to dual boot Arch Linux and a Windows operating system please see **Dual boot with Windows**.

Note:

- **UEFI** systems typically need an **EFI system partition**.
- BIOS systems that are partitioned with **GPT** require a **BIOS boot partition** if **GRUB** is used as the bootloader.

Tip: If using **Btrfs**, subvolumes can be used to imitate partitions. See the **Btrfs#Mounting subvolumes** section.

Single root partition

This scheme is the simplest and should be enough for most use cases. A **swapfile** can be created and easily resized as needed. It usually makes sense to start by considering a single / partition and then separate out others based on specific use cases like RAID, encryption, a shared media partition, etc.

Discrete partitions



This article or section needs expansion.

Reason: List the appropriate [GPT partition type GUIDs](#). (Discuss in [Talk:Partitioning#](#))



Separating out a path as a partition allows for the choice of a different filesystem and mount options. In some cases like a media partition, they can also be shared between operating systems.

Below are some example layouts that can be used when partitioning, and the following subsections detail a few of the directories which can be placed on their own separate partition and then **mounted** at mount points under `/`. See [file-hierarchy \(7\)](#) for a full description of the contents of these directories.

`/`

The [root directory](#) is the top of the hierarchy, the point where the primary filesystem is mounted and from which all other filesystems stem. All files and directories appear under the root directory `/`, even if they are stored on different physical devices. The contents of the root filesystem must be adequate to boot, restore, recover, and/or repair the system.

Therefore, certain directories under `/` are not candidates for separate partitions.

The `/` partition or root partition is necessary and it is the most important. The other partitions can be replaced by it.

Warning: Directories essential for booting (except for `/boot`) **must** be on the same partition as `/` or mounted in early userspace by the [initramfs](#). These essential directories are: `/etc` and `/usr` [\[2\]](#).

`/` traditionally contains the `/usr` directory, which can grow significantly depending upon how much software is installed. 15–20 GiB should be sufficient for most users with modern hard disks. If you plan to store a swap file here, you might need a larger partition size.

`/boot`

The `/boot` directory contains the kernel and ramdisk images as well as the boot loader configuration file and boot loader stages. It also stores data that is used before the kernel begins executing user-space programs. `/boot` is not required for normal system operation, but only during boot and kernel upgrades (when regenerating the initial ramdisk).

Note:

- A separate `/boot` partition is only required if your boot loader is not capable of accessing the `/boot` directory that resides in `/`. For example, if the boot loader does not support that file system or if your `/` is on a stacked block device (e.g. software [RAID](#), a [encrypted volume](#) or a [LVM](#) volume) and the boot loader does not have drivers for it. See [Arch boot process#Boot loader](#) for more information on boot loader requirements and capabilities.
- If booting using an [UEFI boot loader](#) that does not have drivers for other file systems it is recommended to mount the [EFI system partition](#) to `/boot`. See [EFI system partition#Mount the partition](#) for more information.

A suggested size for `/boot` is 200 MiB unless you are using [EFI system partition](#) as `/boot`, in which case at least 260 MiB is recommended.

Warning: File systems can get new features not yet supported by [boot loaders](#), making them unsuitable for a `/boot` partition unless incompatible features remain disabled.

`/home`

The `/home` directory contains user-specific configuration files, caches, application data and media files.

Separating out `/home` allows `/` to be re-partitioned separately, but note that you can still reinstall Arch with `/home` untouched even if it is not separate—the other top-level directories just need to be removed, and then `pacstrap` can be run.

You should not share home directories between users on different distributions, because they use incompatible software versions and patches. Instead, consider sharing a media partition or at least using different home directories on the same `/home` partition. The size of this partition varies.

`/var`

The `/var` directory stores variable data such as spool directories and files, administrative and logging data, [pacman](#)'s cache, etc. It is used, for example, for caching and logging, and hence frequently read or written. Keeping it in a separate partition avoids running out of disk space due to flunky logs, etc.

It exists to make it possible to mount `/usr` as read-only. Everything that historically went into `/usr` that is written to during system operation (as opposed to installation and software maintenance) must reside under `/var`.

Note: `/var` contains many small files. The choice of file system type should consider this fact if a separate partition is used.

`/var` will contain, among other data, the [pacman](#) cache. Retaining these packages is helpful in case a package upgrade causes instability, requiring a [downgrade](#) to an older, archived package. The pacman cache will grow as the system is expanded and updated, but it can be safely cleared if space becomes an issue. 8–12 GiB on a desktop system should be sufficient for `/var`, depending on how much software will be installed.

`/data`

One can consider mounting a "data" partition to cover various files to be shared by all users. Using the `/home` partition for this purpose is fine as well. The size of this partition varies.

Swap

A [swap](#) is a file or partition that provides disk space used as virtual memory. Swap files and swap partitions are equally performant, but swap files are much easier to resize as needed. A swap partition can *potentially* be shared between operating systems, but not if hibernation is used.

Historically, the general rule for swap partition size was to allocate twice the amount of physical RAM. As computers have gained ever larger memory capacities, this rule is outdated. For example, on average desktop machines with up to 512 MiB RAM, the 2× rule is usually adequate; if a sufficient amount of RAM (more than 1024 MiB) is available, it may be possible to have a smaller swap partition.

To use hibernation (a.k.a suspend to disk) it is advised to create the swap partition at the size of RAM. Although the kernel will try to compress the suspend-to-disk image to fit the swap space there is no guarantee it will succeed if the used swap space is significantly smaller than RAM. See [Power management/Suspend and hibernate#Hibernation](#) for more information.

Example layouts



This article or section needs expansion.

Reason: Improve current examples. (Discuss in [Talk:Partitioning#Table draft 2](#))



The following examples use `/dev/sda` as the example disk with `/dev/sda1` as the first partition. The block device naming scheme will differ if you are partitioning a [NVMe](#) disk (e.g. `/dev/nvme0n1` with partitions starting from `/dev/nvme0n1p1`) or an SD card or eMMC disk (e.g. `/dev/mmcblk0` with partitions starting from `/dev/mmcblk0p1`). See [Device file#Block device names](#) for more information.

Note:

- UEFI booting does not involve any "boot" flag, booting relies solely on the boot entries in NVRAM. [Parted](#) and its front-ends use a "boot" flag on GPT to indicate that a partition is an EFI system partition.

- There is no requirement to have all required/wanted partitions on the same disk or to use the same type of partition table for all disks.

UEFI/GPT layout example

Mount point on the installed system	Partition	Partition type GUID	Partition attributes	Suggested size
/boot or /efi ¹	/dev/sda1	C12A7328-F81F-11D2-BA4B-00A0C93EC93B : EFI system partition		At least 260 MiB
[SWAP]	/dev/sda2	0657FD6D-A4AB-43C4-84E5-0933C84B4F4F : Linux swap		More than 512 MiB
/	/dev/sda3	4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709 : Linux x86-64 root (/)		Remainder of the device

BIOS/MBR layout example

Mount point on the installed system	Partition	Partition type ID	Boot flag	Suggested size
[SWAP]	/dev/sda1	82 : Linux swap	No	More than 512 MiB
/	/dev/sda2	83 : Linux	Yes	Remainder of the device
N/A	<i>Unallocated space</i> ²	N/A	N/A	At least 16.5 KiB at the end of the disk

BIOS/GPT layout example

Mount point on the installed system	Partition	Partition type GUID	Partition attributes	Suggested size
None	/dev/sda1	21686148-6449-6E6F-744E-656564454649 : BIOS boot partition ³		1 MiB
[SWAP]	/dev/sda2	0657FD6D-A4AB-43C4-84E5-0933C84B4F4F : Linux swap		More than 512 MiB
/	/dev/sda3	4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709 : Linux x86-64 root (/)		Remainder of the device

1. The ESP can be mounted to `/efi` if the used boot loader is capable of accessing the file system (and everything above it) on which the kernel and initramfs images are located. See [EFI system partition#Typical mount points](#) and the warning in [Arch boot process#Boot loader](#) for details.
2. An unpartitioned space of at least 33 512-byte sectors (16.5 KiB) at the end of the disk to allow [converting to GPT](#) in the future. The space will be required for the backup GPT header. The recommendation to preserve an unpartitioned space applies to all MBR partitioned disks.
3. A BIOS boot partition is only required when using [GRUB](#) for BIOS booting from a GPT disk. The partition has nothing to do with `/boot`, and it must not be formatted with a file system or mounted.

Tools

Partitioning tools

The following programs are used to create and/or manipulate device partition tables and partitions. See the linked articles for the exact commands to be used.

This table will help you to choose utility for your needs:

	MBR	GPT
Dialog	fdisk parted	fdisk gdisk parted
Pseudo-graphics	cdisk	cdisk cgdisk
Non-interactive	sfdisk parted	sfdisk sgdisk parted
Graphical	GParted gnome-disk-utility partitionmanager	GParted gnome-disk-utility partitionmanager

Warning: To partition devices, use a partitioning tool compatible to the chosen type of partition table. Incompatible tools may result in the destruction of that table, along with existing partitions or data.

fdisk

fdisk and its related utilities are described in the [fdisk](#) article.

- [fdisk \(util-linux\)](#)
 - [fdisk \(8\)](#) – Dialog-driven program for creation and manipulation of partition tables.
 - [cdisk \(8\)](#) – [Curses](#)-based variant of fdisk.
 - [sfdisk \(8\)](#) – Scriptable variant of fdisk.

GPT fdisk

gdisk and its related utilities are described in the [gdisk](#) article.

- [GPT fdisk \(gptfdisk\)](#)
 - [gdisk \(8\)](#) – Interactive [GUID partition table \(GPT\)](#) manipulator.
 - [cgdisk \(8\)](#) – Curses-based variant of gdisk.
 - [sgdisk \(8\)](#) – Scriptable variant of gdisk.

GNU Parted

These group of tools are described in the [GNU Parted](#) article.

- [GNU Parted](#) — Terminal partitioning tool.
<https://www.gnu.org/software/parted/parted.html> || [parted](#)
- [GNOME Disks](#) — Disk management utility for GNOME.
<https://wiki.gnome.org/Apps/Disks> || [gnome-disk-utility](#)
- [GParted](#) — GTK partition editor for graphically managing your disk partitions.
<https://gparted.sourceforge.io/> || [gparted](#)
- [KDE Partition Manager](#) — Utility program for KDE to manage the disk devices, partitions and file systems.

Backup

- [fdisk](#) can create a backup of the partitions table. See [fdisk#Backup and restore partition table](#).
- [GPT fdisk](#) can create a binary backup consisting of the protective MBR, the main GPT header, the backup GPT header, and one copy of the partition table. See [GPT fdisk#Backup and restore partition table](#).

Recovery

- [gpart](#) — A utility that guesses the contents of a destroyed MBR partition table. Its usage is explained in the [gpart\(8\)](#) man page.
<https://github.com/baruch/gpart> || [gpart](#)
- [GPT fdisk](#) — A partitioning tool that can restore the primary GPT header (located at the start of the disk) from the secondary GPT header (located at the end of the disk) or vice versa.
<https://www.rodsbooks.com/gdisk/> || [gptfdisk](#)
- [TestDisk](#) — A utility that supports recovering lost partitions on both MBR and GPT.
<https://www.cgsecurity.org/index.html?testdisk.html> || [testdisk](#)

Partition alignment

[fdisk](#), [gdisk](#) and [parted](#) handle alignment automatically. See [GNU Parted#Check alignment](#) if you want to verify your alignment after partitioning.

For certain drives [Advanced Format](#) might be able to provide a better-performing alignment.

GPT kernel support

The `CONFIG_EFI_PARTITION` option in the kernel config enables GPT support in the kernel (despite the name, EFI PARTITION). This option must be built in the kernel and not compiled as a loadable module. This option is required even if GPT disks are used only for data storage and not for booting. This option is enabled by default in all Arch's [officially supported kernels](#). In case of a custom kernel, enable this option by doing `CONFIG_EFI_PARTITION=y`.

Troubleshooting



This article or section needs expansion.

Reason: Add instructions for changing the drive controller operating mode from firmware RAID to native (i.e. AHCI for SATA) (Discuss in [Talk:Installation guide#Boot issues faced when installing on modern machines.](#))



Tricking old BIOS into booting from GPT

Some old BIOSes (from before year 2010) attempt to parse the boot sector and refuse to boot it if it does not contain a bootable MBR partition. This is a problem if one wants to use GPT on this disk, because, from the BIOS viewpoint, it contains only one, non-bootable, MBR partition of type `ee` (i.e., the protective MBR partition). One can mark the protective MBR entry as bootable using `fdisk -t mbr /dev/sda`, and it will work on some BIOSes. However, the UEFI specification prohibits the protective MBR partition entry from being bootable, and UEFI-based boards do care about this, even in the legacy boot mode. So, this matters if one wants to create a GPT-based USB flash drive that is supposed to boot both on modern UEFI-based boards and also on old BIOSes that insist on finding a bootable MBR partition. It is not possible to solve this problem using traditional tools such as [fdisk](#) or [gdisk](#), but it is possible to create a fake MBR partition entry suitable for both kinds of BIOSes manually as a sequence of bytes.

The command below will overwrite the second MBR partition slot and add a bootable partition there of type 0 (i.e. unused), covering only the first sector of the device. It will not interfere with the GPT or with the first MBR partition entry which normally contains a protective MBR partition.

```
# printf '\x200\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0\x0' | dd of=/dev/sda bs=1 seek=462
```

The end result will look like this:

```
# fdisk -t mbr -l /dev/sda

Disk /dev/sda: 232.9 GiB, 250059350016 bytes, 488397168 sectors
Disk model: ST3250820AS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device      Boot Start      End  Sectors  Size Id Type
/dev/sda1             1 488397167 488397167 232.9G ee GPT
/dev/sda2    *           0          0           1   512B  0 Empty

Partition table entries are not in disk order.
```

See also

- [Wikipedia:Disk partitioning](#)
- [Wikipedia:Binary prefix](#)
- [Understanding Disk Drive Terminology](#)
- [What is a Master Boot Record \(MBR\)?](#)
- Rod Smith's page on [What's a GPT?](#) and [Bootng OSes from GPT](#)
- [Make the most of large drives with GPT and Linux - IBM Developer](#)
- [Microsoft's Windows and GPT FAQ](#)
- [Partition Alignment](#) (with examples)

Categories: [File systems](#) | [Boot process](#) | [System recovery](#)